FACULTY OF ARTS

KU LEUVEN

Computational Linguistics

Lecture 3: Language models and word meaning

Tim Van de Cruys



Tuesday 6 October 2026

Tokenization

- Text is encoded by a computer as a sequence of characters (string)
- Words, spaces, and punctuation are encoded in the same way
- We need to define and mark word boundaries: **tokenization**

Tokenization

What is a word?

A consecutive sequence of characters, separated by whitespace (space, newline) and/or punctuation?

- Apostrophe? *sms'en, gsm'etje, doin'*

Tokenization

What is a word?

A consecutive sequence of characters, separated by whitespace (space, newline) and/or punctuation?

- Apostrophe? *sms'en, gsm'etje, doin'*
- Hyphen? *self-driving, what-do-you-mean-look*

Tokenization

What is a word?

A consecutive sequence of characters, separated by whitespace (space, newline) and/or punctuation?

- Apostrophe? *sms'en, gsm'etje, doin'*
- Hyphen? *self-driving, what-do-you-mean-look*
- And if there is no separation?
  - langeafstandsloper, serotonineheropnameremmer
  - Chinese script: a sequence of characters without whitespace
  - Agglutinating languages: Turkish, Basque, Inuit languages

Tokenization

What is a word?

- Proper names: San Francisco, New York

Tokenization

What is a word?

- Proper names: San Francisco, New York
- Fixed expressions:
  - by and large, once and for all
  - piece of cake, spill the beans
  - et cetera, wishful thinking

Tokenization

What is a word?

- Proper names: San Francisco, New York
- Fixed expressions:
  - by and large, once and for all
  - piece of cake, spill the beans
  - et cetera, wishful thinking
- Modern neural models avoid the question: text is split into frequency-based **subword units** (e.g. byte pair encoding), which may be smaller than words; more on this later in the course

Sentence detection

- Certain punctuation characters are unambiguous (‘?’, ‘!’)
- A dot (‘.’) is very ambiguous
  - *Obama held his final address to the U.N. General Assembly on Wednesday.*
  - *Different genres such as the motet, the cantata, etc. developed throughout the centuries.*
- Context is important to determine where a sentence ends

Language models

- Symbolic models (rule-based) give a hard, binary model of well-formed sentences
- For many natural language processing tasks, a probabilistic model of a language (what is the probability of a string being well-formed?) is more useful
- To specify a correct probability distribution, the probability of all sentences in a language must sum to 1

Uses of language models

- Speech recognition
  - ‘I ate a tomato’ is more likely than ‘eye eight atom eight oh’
- OCR & Handwriting recognition
  - More probable sentences are more likely correct readings
- Machine translation
  - More likely sentences are probably better translations
- Generation
  - More likely sentences are probably better natural language generations
- Context sensitive spelling correction
  - ‘Their is a problem wit this sentence’

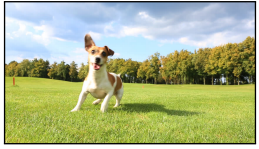
Completion prediction

- Remember the guessing game of lecture 1: a language model predicts the next word in the sentence
  - Please turn off your mobile ____
  - If he loses, he has only himself to ____
- Predictive text input systems can guess what you are typing and provide choices on how to complete it

Statistical language model

- How to compute the probability of a sentence, e.g. *the dog was running across the green field* ?
- We could count the number of times we attest the sentence, and divide by the total number of sentences in our corpus

$$\frac{count(\text{the dog was running across the green field})}{total\ number\ of\ sentences}$$



Statistical language model

- How to compute the probability of a sentence, e.g. *the dog was running across the green field* ?
- We could count the number of times we attest the sentence, and divide by the total number of sentences in our corpus

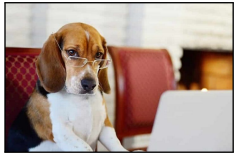
$$\frac{\textit{count}(\text{the dog was running across the green field})}{\textit{total number of sentences}} = \frac{0}{10^{14}} = 0$$



Statistical language model

How to compute $p(\text{the,dog,was,running,across,the,green,field})$?

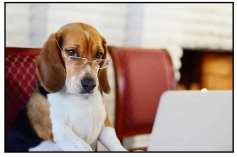
- We compute the joint probability by decomposing into conditional probabilities
- $p(\text{the}) \times p(\text{dog}|\text{the})$
  $\times\ p(\text{was}|\text{the dog})$
  $\times\ p(\text{running}|\text{the dog was})$
  $\times\ p(\text{across}|\text{the dog was running})$
  $\times\ p(\text{the}|\text{the dog was running across})$
  $\times\ p(\text{green}|\text{the dog was running across the})$
  $\times\ p(\text{field}|\text{the dog was running across the green})$



Statistical language model

How to compute p(the,dog,was,running,across,the,green,field) ?

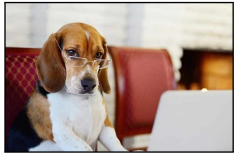
- We compute the joint probability by decomposing into conditional probabilities
- p(the) $\times$ p(dog|the)
  $\times$ p(was|the dog)
  $\times$ p(running|~~the~~ dog was)
  $\times$ p(across|~~the dog~~ was running)
  $\times$ p(the|~~the dog was~~ running across)
  $\times$ p(green|~~the dog was running~~ across the)
  $\times$ p(field|~~the dog was running across~~ the green)
- ... using only a limited context



Statistical language model

How to compute p(the,dog,was,running,across,the,green,field) ?

- We compute the joint probability by decomposing into conditional probabilities
- p(the) $\times$ p(dog|the)
  $\times$ p(was|the dog)
  $\times$ p(running|dog was)
  $\times$ p(across|was running)
  $\times$ p(the|running across)
  $\times$ p(green|across the)
  $\times$ p(field|the green)
- ... using only a limited context



Estimating probabilities

$$p(\text{the}) = \frac{count(the)}{count(*)}$$

$$p(\text{dog}|\text{the}) = \frac{count(the,dog)}{count(the)}$$

$$p(\text{green}|\text{across the}) = \frac{count(across,the,green)}{count(across,the)}$$

N-gram models

- Estimate probability of each word given prior context
- Multiplication of individual probabilites gives sentence probability
- An n-gram model uses only n–1 words of prior context
  - unigram: $p(\text{dog})$
  - bigram: $p(\text{dog} \mid \text{the})$
  - trigram: $p(\text{green} \mid \text{across the})$
  - ... and so on
- N-gram models are based on the **Markov assumption**
  - Future behavior of a dynamical system only depends on its recent history
  - In a kth-order Markov model, the next state only depends on the k most recent states
  - N-gram model is a (n–1)th-order Markov model.

Part of speech

- Words can be categorised in classes that behave similarly
  1. I can see a _____.
  2. The _____ is on the table.
  3. She's thinking about a _____.
- Part of speech provides information about the context in the sentence
  - *adjective* often succeeded by *noun*
  - *personal pronoun* often succeeded by *verb*
- Useful information for language processing applications
- Automatically assigning part of speech: **part of speech tagging**



Named entity recognition

- An important subtask in NLP
- Find and classify the names in a text

According to a recent survey published by Eurostat on Wednesday 28 October, the gap between the hourly cost of labour in Paris and in Berlin is tightening.

Named entity recognition

- An important subtask in NLP
- Find and classify the names in a text

According to a recent survey published by Eurostat on Wednesday 28 October, the gap between the hourly cost of labour in Paris and in Berlin is tightening.

Named entity recognition

- An important subtask in NLP
- Find and classify the names in a text

According to a recent survey published by Eurostat on Wednesday 28 October, the gap between the hourly cost of labour in Paris and in Berlin is tightening.

- person
- location
- date
- organization

Named entity recognition

IOB Encoding

```
published   O
by          O
Eurostat    B-ORG
on          O
Wednesday   B-DAT
28          I-DAT
October     I-DAT
```

- Using a suitable encoding scheme, named entity recognition becomes a sequence labeling task
- **I**nside, **O**utside, **B**eginning
- B-XX in order to separate adjacent entities of the same type

Manual definition of features

- Words
  - Current word
  - Previous/next word (context)
  - Substring of characters
  - Word form
- Additional linguistic annotations
  - POS
- Context labels
  - Previous/next label

Paramedics	treat	woman	and	boy
after	crash	in	Stoke-on-Trent	.

Manual definition of features

- Words
  - Current word
  - Previous/next word (context)
  - Substring of characters
  - Word form
- Additional linguistic annotations
  - POS
- Context labels
  - Previous/next label

Paramedics treat woman and boy

after crash in Stoke-on-Trent .

Manual definition of features

- Words
  - Current word
  - Previous/next word (context)
  - Substring of characters
  - Word form
- Additional linguistic annotations
  - POS
- Context labels
  - Previous/next label

Paramedics	treat	woman	and	boy
after	crash	in	Stoke-on-Trent	.

Manual definition of features

- Words
  - Current word
  - Previous/next word (context)
  - Substring of characters
  - Word form
- Additional linguistic annotations
  - POS
- Context labels
  - Previous/next label

Paramedics	treat	woman	and		boy
after	crash	in	Stoke-on-Trent	.	

Manual definition of features

- Words
  - Current word
  - Previous/next word (context)
  - Substring of characters
  - Word form
- Additional linguistic annotations
  - POS
- Context labels
  - Previous/next label

Paramedics	treat	woman	and	boy
after	crash	in	Xx-x-Xx	.

Manual definition of features

- Words
  - Current word
  - Previous/next word (context)
  - Substring of characters
  - Word form
- Additional linguistic annotations
  - POS
- Context labels
  - Previous/next label

Paramedics	treat	woman	and	boy
NNS	VBP	NN	CC	NN
after	crash	in	Stoke-on-Trent	.
IN	NN	IN	NNP	PUNCT

Manual definition of features

- Words
  - Current word
  - Previous/next word (context)
  - Substring of characters
  - Word form
- Additional linguistic annotations
  - POS
- Context labels
  - Previous/next label

Paramedics	treat	woman	and	boy
NNS	VBP	NN	CC	NN
O	O	O	O	O
after	crash	in	Stoke-on-Trent	.
IN	NN	IN	NNP	PUNCT
O	O	O	?	O

Manual definition of features

- Words
  - Current word
  - Previous/next word (context)
  - Substring of characters
  - Word form
- Additional linguistic annotations
  - POS
- Context labels
  - Previous/next label

Paramedics	treat	woman	and	boy
NNS	VBP	NN	CC	NN
O	O	O	O	O
after	crash	in	Stoke-on-Trent	.
IN	NN	IN	NNP	PUNCT
O	O	O	?	O

Sequence tagging

- Labeling a word in a sequence is the same as a simple classification
- We have predicted labels for previous words
- We use the predicted labels of previous words, as well as observed features (based on current, previous, and future words) in order to predict the best possible label for the current word

crash	in	Stoke-on-Trent	.
NN	IN	NNP	PUNCT
O	O	?	?

Features	
w_0	Stoke-on-Trent
w_{-1}	in
w_{+1}	.
pos_0	NNP
pos_{-1}	IN
pos_{+1}	PUNCT
hasHyphen	True
$label_{-1}$	O

Global optimization

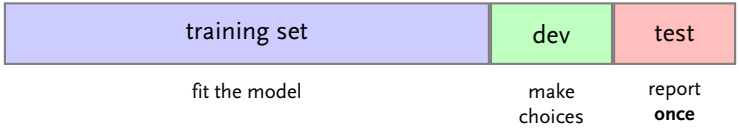
- Problem: we perform a series of local judgements, without taking into account the global optimality
- We hope that the local conditions give us a global optimum
- To solve our problem:
  - Instead of emitting a single answer, predict the *n* best answers
  - This way, we can compute the best global sequence

Experiment

- We have three sentiment classifiers
  - a list of hand-written **rules** (symbolic)
  - **naive Bayes** (statistical)
  - **logistic regression** (statistical)

- Which one is better? how can we find out?
- Common procedure in NLP: **experiment**

Train, development, test



- A model must be evaluated on data it has **never seen**: we care about generalization, not memorization
- **Training** set: estimate the parameters (probabilities, weights)
- **Development** (validation) set: for every choice you make (features, smoothing, thresholds, which model to keep: hyperparameters)
- **Test** set: touched once, at the very end, for the number you report
- Typical splits: 80/10/10 (train/dev/test)
- Little data? **Cross-validation** on the training part; the test set still stays apart

Leakage

- **Leakage**: information about the test set finds its way into the model, so the reported score is too optimistic
- It rarely looks like cheating:
  - you tried ten settings and kept the one that was best *on the test set*
  - the vocabulary was computed on *all* the data, before splitting
  - the same review (or a near-duplicate) occurs in both training and test
  - texts by the same author are included in both training and test
- Everything that is *learned* or *chosen* must be learned or chosen without looking at the test set

Baseline

- *'Our model reaches 95% accuracy'*: is that good?
- Example: toxic comment detection
  - about 95% of the comments are *not* toxic
  - A model that always answers `not toxic` scores 95%, while it does not find a single toxic comment
- A number only means something next to a **baseline**
  - majority class (always predict the most frequent label)
  - a handful of simple rules, a word list
  - the simplest model that could possibly work
- If your sophisticated model does not beat the simple baseline, you have a negative result

Upper bound

- Opposite of the baseline: the best score that can be expected
- Usually **human performance**: inter-annotator agreement
  - two annotators label the same items independently
  - percentage of agreement, or agreement corrected for chance (e.g. Krippendorff's α)
- The gold standard is only as reliable as the agreement between annotators
  - e.g. sentiment analysis: annotators often agree on 80–90% of the items
  - subjective tasks (irony, toxicity): considerably lower
- A score is interpreted between baseline and upper bound

Beyond accuracy

Confusion matrix

		gold standard	
		toxic	not toxic
system	toxic	true positive (TP)	false positive (FP)
	not toxic	false negative (FN)	true negative (TN)

$$P = \frac{TP}{TP + FP} \qquad R = \frac{TP}{TP + FN} \qquad F_1 = \frac{2PR}{P + R}$$

- The always-`not toxic` baseline: accuracy 95%, but recall and F_1 for the class *toxic*: 0%

Multiple classes

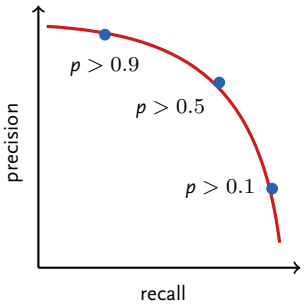
- With more than two classes: compute P, R, F_1 per class, then average
  - **micro**-average: pool all decisions $\rightarrow$ dominated by the frequent classes
  - **macro**-average: mean of the per-class scores $\rightarrow$ every class counts equally
- With imbalanced classes, both averages can differ considerably

Performance on sequences

- How to evaluate named entity recognition?
- Precision, recall and F-measure, computed over **entities** rather than words
  - precision: percentage of entities returned by the system that are correct
  - recall: percentage of the entities in the test set that are found by the system
- An entity is correct if both its boundaries and its type correspond to the test set
  - *Charles Michel* as PER, but only *Michel* tagged: incorrect
  - *European Council* tagged as LOC: incorrect
- Per-word accuracy would be misleadingly high, as most words are labeled O

Thresholds

- Naive Bayes and logistic regression output a **probability**
- A decision requires a **threshold**, e.g. toxic if $p > 0.5$
  - lower threshold: higher recall, lower precision
  - higher threshold: higher precision, lower recall
- The threshold is a hyperparameter: it is set on the development set



Significance

- Example: naive Bayes 84.2% accuracy, logistic regression 84.7% accuracy
- Is logistic regression better? Not necessarily
  - the test set is a sample: a different set of 1,000 reviews yields a different score
  - models with random components (initialization, order of the data) yield different results with different random seeds
- Guidelines:
  - use a sufficiently large test set; report size
  - repeat with different seeds or folds; report mean and standard deviation
  - small differences between models are often not significant: use a significance test (e.g. McNemar's test, paired bootstrap)

Error analysis

- An evaluation score indicates how often a model makes mistakes, not why
- Error analysis: inspection of the errors on the development set
  - take a sample of errors (e.g. 50) and categorize them
  - which categories are frequent, and which could be addressed?
- Linguistic knowledge is often helpful
- Goal: explain a pattern of errors

Checklist

1. Keep a separate test set, and use it only once
2. Avoid leakage
3. Compare against a baseline
4. Choose metrics that suit the task and the class distribution
5. Make all choices (incl. thresholds) on the development set
6. Report the uncertainty of the results
7. Analyze the errors
8. Support every claim with results